ENGINEERING HANDOFF / TEAMMATE GUIDE

Intelligent Multi-Cloud Security Posture Management

Frontend & Backend Build Guide

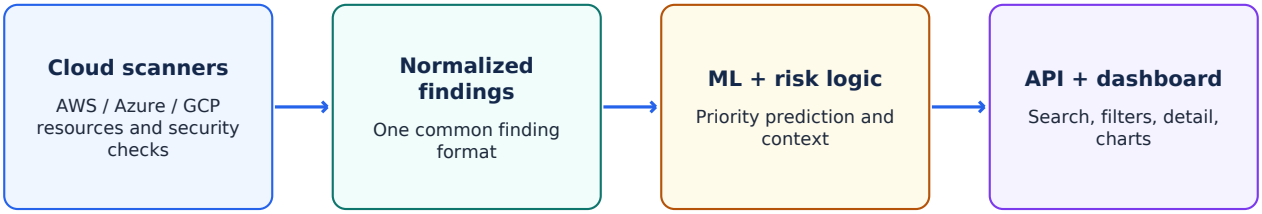
A practical guide for a developer who is new to ML: what the model does, which fields it needs, what APIs the app should expose, and how the dashboard, backend, and security intelligence fit together.

Prepared for: Frontend / Backend implementation teammate
Project ML lead: Rutik Bhendarkar
Assistant: Fridat (Gemini-powered, read-only project copilot)

IMPORTANT: This document separates existing notebook artifacts from proposed API and application work. It is a handoff design, not a claim that every service or live cloud integration is already deployed.

1. Start here: what are we building?

The project collects cloud security findings, adds context, uses a trained ML model to predict a priority label, and presents useful information to a security analyst. The frontend should not need to understand how Random Forest works. It should call the backend and render stable JSON responses.



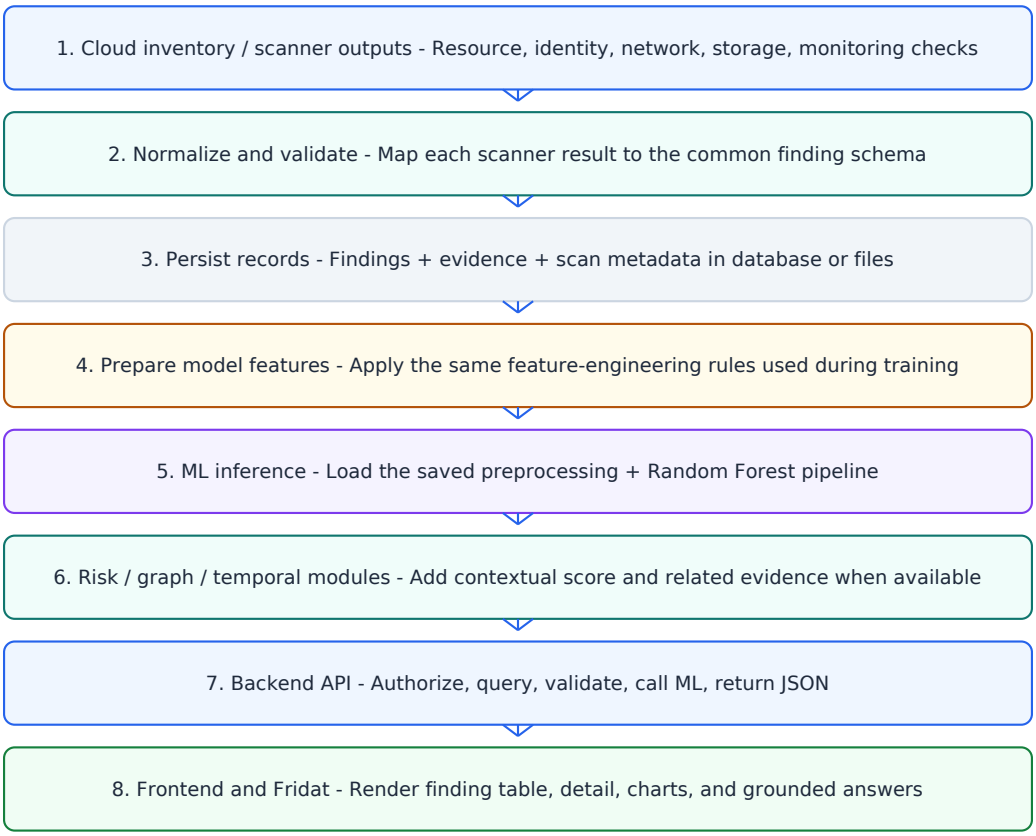
The high-level layers

Layer	Main responsibility	Frontend/backend implication
Multi-cloud collection	Provider connectors, resource discovery, inventory	Scanners should export the common schema. Live cloud connectors are a separate workstream.
CSPM detection	IAM, network, data, logging, and compliance checks	Each scanner creates findings with a stable ID, severity, evidence, and resource identity.
Findings/data layer	Central storage of findings and history	Backend owns data access, validation, filtering, pagination, and persistence.
AI/ML intelligence	Predicted priority, contextual risk, correlation, recommendations	Model is called by server-side code; browser should not load the serialized model.
Presentation / alerts	Dashboard, reports, optional notifications, Fridat assistant	Frontend visualizes API responses; Gemini key remains on server/Colab only.

Implementation status: The ML notebook already defines a Random Forest pipeline, evaluation, prediction outputs, feature-importance output, candidate correlations, and blast-radius summaries. Several API routes in this guide are proposals for the app team to implement. CSV or notebook outputs are not the same as a deployed REST backend.

2. End-to-end architecture and flow

Use this as the shared mental model. The ML notebook is currently a training/analysis workspace. The web application should consume model outputs through a backend service or a reproducible inference module.



Key rule: the model only works reliably when inference uses the same feature names and equivalent feature-engineering logic as training. Do not recreate the model formula in JavaScript.

Simple request lifecycle

Analyst opens a dashboard page -> browser calls an API endpoint -> backend validates filters and loads records -> backend optionally runs model inference -> backend returns JSON -> frontend renders the response. The browser should never query CSVs directly or receive the Gemini API key.

3. Who owns what?

Workstream	Primary owner / input	Expected integration contract
Infrastructure & network	Network/infrastructure teammate: compute, VPC/VNet, security groups, NSGs, public exposure	Finding records + resource IDs + evidence + scan timestamp
IAM security	IAM teammate: roles, policies, MFA, least privilege, excessive access, trust relationships	Finding records that name the affected identity/resource and evidence
Data, monitoring & compliance	Data teammate: public storage, encryption, logging, CIS-style checks	Finding records + control/check identifiers + evidence
ML security intelligence	Rutik: feature engineering, priority prediction, risk scoring, correlation, analysis outputs	Model-ready feature row -> predicted priority/confidence and optional probability columns
Backend teammate	API routes, input validation, persistence, authentication/authorization, model loading, retrieval for Fridat	Stable, versioned JSON contracts; no cloud writes without separately approved scope
Frontend teammate	Dashboard pages, filters, tables, detail views, charts, chatbot UI, loading/error states	Calls API; does not train or reimplement the ML model

Minimum common finding record

Every scanner should provide a stable `finding_id`, a `resource_id`, `cloud`, `region` where relevant, `category`, `resource_type`, `finding`, `evidence`, `severity`, `status`, and `detected_at` or an equivalent timestamp. Optional fields may be null, but a missing value must not be silently converted to “safe.”

> Agree before coding: choose one canonical spelling for field names and timestamps. If one scanner sends `resourceId` and another sends `resource_id`, normalize that difference in the backend adapter instead of adding special cases to the UI.

A stable ID is essential: it links a finding in the dashboard to model predictions, evidence, relationship records, history, and remediation tracking.

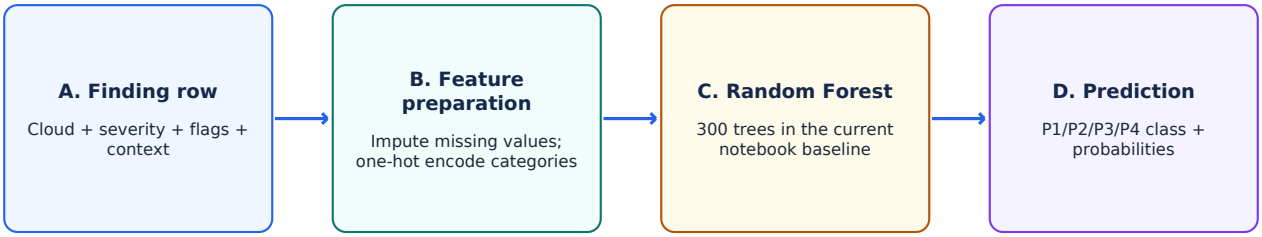
4. ML explained without ML jargon

Feature = an input detail. For example: cloud provider, severity, internet exposure, privileged-access context, or resource type.

Target = the answer the model learns to predict. In this notebook, the target column is the existing CSPM field `priority`. It is not included as an input feature.

Training = examples used to learn patterns. The notebook fits a preprocessing pipeline and a Random Forest classifier using training rows.

Prediction = the model output for a new row. The trained pipeline returns `ml_predicted_priority`, class probability columns, and `ml_confidence` (the largest class probability). This confidence is not a guarantee that the security assessment is correct.



What the frontend should show

UI element	Field / source	How to label it
Recorded severity	`severity`	Show as the scanner's recorded severity. Do not overwrite it with ML priority.
ML priority prediction	`ml_predicted_priority`	Label "Predicted priority" or "ML priority," not "confirmed severity."
Model confidence	`ml_confidence`	Optional percentage. Tooltip: model confidence is not security certainty.
Class probabilities	`prob_P1`, `prob_P2`, `prob_P3`, `prob_P4` when present	Optional probability breakdown. Only display returned fields.
Contextual risk score	`contextual_risk_score` when available	Keep separate from ML prediction; it may be a heuristic score.
Evidence	`evidence`	Show as observed/reported evidence; do not ask Gemini to invent missing evidence.

Important: the notebook uses priority labels in the order P1, P2, P3, P4. Confirm with the ML lead/product owner which label means highest urgency before displaying user-facing names such as Critical/High/Medium/Low.

5. Model feature dictionary (part 1)

The feature list below is taken from the current ML notebook. These fields are inputs to the model; they do not all need to be visible as columns in the UI. The backend/inference function must supply the expected fields after applying feature engineering.

Feature name	Plain-language meaning	Group
`cloud`	Provider label, such as AWS, Azure, or GCP.	Source context
`region`	Cloud region associated with the finding.	Source context
`category`	Finding category, such as network, IAM, storage, or monitoring.	Source context
`resource_type`	Type of resource affected, such as compute, storage, or identity.	Source context
`severity`	Severity already recorded by the scanner or dataset.	Source context
`internet_exposed`	Whether the finding/resource is flagged as internet exposed.	Security context
`privileged_access_related`	Whether privileged access is related to the finding.	Security context
`business_critical`	Whether the resource is flagged as business critical.	Security context
`sensitive_data_related`	Whether sensitive data is related to the finding.	Security context
`correlation_group`	Group label used to associate related findings. Not proof of a direct connection.	Correlation context
`finding_age_days`	Age of a finding measured in days. Requires a reliable detection time/reference time.	Temporal
`detection_delay_days`	Delay between relevant event/source time and detection, as defined in feature engineering.	Temporal
`detection_year`	Year derived from detection timestamp.	Temporal
`detection_month`	Month derived from detection timestamp.	Temporal
`detection_dayofweek`	Day of week derived from detection timestamp.	Temporal
`detection_hour`	Hour derived from detection timestamp.	Temporal
`detected_on_weekend`	Whether detection timestamp falls on weekend.	Temporal
`severity_encoded`	Numeric representation of recorded severity. Mapping must match training.	Engineered

5. Model feature dictionary (part 2)

Feature name	Plain-language meaning	Group
`exposure_factor_count`	Count of defined exposure-related flags/factors. Use the same formula as training.	Engineered
`internet_business_risk`	Interaction feature combining internet exposure and business criticality.	Engineered
`internet_sensitive_risk`	Interaction feature combining internet exposure and sensitive-data context.	Engineered
`internet_privileged_risk`	Interaction feature combining internet exposure and privileged-access context.	Engineered
`privileged_business_risk`	Interaction feature combining privilege and business criticality.	Engineered
`privileged_sensitive_risk`	Interaction feature combining privilege and sensitive-data context.	Engineered
`high_impact_context`	Combined context flag built by feature engineering. Exact rule must be reused from notebook.	Engineered
`exposure_type`	Derived exposure category.	Engineered
`identity_risk_type`	Derived identity/access-risk category.	Engineered
`data_risk_type`	Derived data-risk category.	Engineered
`critical_privileged_resource`	Combined flag for critical resource plus privileged access.	Engineered
`critical_sensitive_resource`	Combined flag for critical resource plus sensitive data.	Engineered
`critical_internet_resource`	Combined flag for critical resource plus internet exposure.	Engineered
`finding_age_category`	Bucketed category for the age of the finding.	Engineered
`finding_text_length`	Character length of the finding description; not the semantic meaning of the text.	Text metadata
`evidence_text_length`	Character length of evidence text; not a validation that the evidence is correct.	Text metadata

Do not include as model input: the notebook target `priority`, the existing `priority_score`, IDs (`finding_id`, `resource_id`), recommendation text, or post-prediction outputs unless the ML lead deliberately redesigns and revalidates the model. Reuse the saved pipeline rather than training a second model in the backend.

Backend implementation note

Feature engineering happens before the saved pipeline is called. If a feature is derived (age, timestamp parts, interaction flags, or category buckets), create it using the same rules as the training notebook. The safest first integration is one shared Python preprocessing module used both by batch notebook evaluation and API inference.

6. Data contract: what moves between services?

The notebook writes `cspm_ml_predictions.csv`. It preserves original finding fields where present and adds model outputs. It also writes `cspm_feature_importance.csv` and saves the trained pipeline as `cspm_priority_baseline_model.pkl`.

Artifact	Purpose	Consumer
`cspm_findings_feature_engineered.csv`	Master rows after feature engineering	ML pipeline / analyst review
`cspm_priority_baseline_model.pkl`	Fitted preprocessing + Random Forest pipeline	Backend model inference process
`cspm_ml_predictions.csv`	Original fields plus `ml_predicted_priority`, `ml_confidence`, and `prob_*` columns	Backend/dashboard
`cspm_feature_importance.csv`	Global feature-importance table (`feature`, `importance`, `importance_percent`)	Analytics page or “model insights” UI
`cspm_candidate_attack_paths.csv`	Candidate correlations based on available dataset fields	Graph page; must be labelled as candidate/unverified
`cspm_blast_radius_analysis.csv`	Correlation-group impact summary	Risk/graph page; not proven blast radius

Example response shape (illustrative contract, not a live API response)

```
{
"finding_id": "FINDING_ID_FROM_DATA",
"resource_id": "RESOURCE_ID_FROM_DATA",
"cloud": "AWS",
"severity": "high",
"priority": "P2",
"ml_predicted_priority": "P1",
"ml_confidence": 0.82,
"prob_P1": 0.82,
"internet_exposed": true,
"evidence": "Evidence text from the finding record",
"recommendation": "Recorded or separately generated remediation guidance",
"source": "cspm_ml_predictions.csv"
}
```

Do not use this placeholder as real security data. Real IDs, cloud provider values, and evidence must come from the stored records. The backend should return `null` or omit unavailable fields rather than invent values.

7. Proposed backend API contract

These endpoints are a recommended contract for the frontend/backend team. They are not claimed to exist in the notebook. Implement and document them in FastAPI (or the team's agreed backend framework). Keep filtering, sorting, pagination, and model access on the server.

Method + route	Purpose	Example query/body	Response expectation
`GET /api/health`	Service health/version	None	`status`, `version`, optional dependency status
`GET /api/overview`	Dashboard KPI cards	Optional scan/source filter	Finding counts by severity/priority, scan time, data coverage
`GET /api/findings`	Search/list findings	`cloud`, `severity`, `priority`, `status`, `q`, `page`, `page_size`, `sort`	`items[]`, `total`, `page`, `page_size`
`GET /api/findings/{finding_id}`	Finding detail	Path ID	Original record, evidence, ML output if available, source metadata
`POST /api/predictions`	Run model on one or more model-ready finding rows	`{ "finding_ids": ["..."] }`	Prediction fields, model version, warnings for missing features
`GET /api/analytics/priority`	Charts and distribution	Optional cloud/time filter	Counts by recorded priority, predicted priority, severity
`GET /api/resources/{resource_id}/relationships`	Graph resource detail	Path ID	Only known edges with source/evidence/verification state
`GET /api/temporal/events`	History and scan changes	`scan_id`, `event_type`, date range	Events with scan IDs and completeness metadata
`GET /api/remediations`	List remediation tracking records	`status`, `finding_id`	Tracked status and timestamps; separate from actual cloud state
`GET /api/ai-spm/assessments`	AI workload/control assessment	`ai_asset_id`, `status`	Coverage, PASS/FAIL/UNKNOWN/NOT_APPLICABLE, evidence metadata
`POST /api/copilot/chat`	Ask Fridat about project data	`{ "question": "..." }`	`answer`, `citations[]`, `retrieved_sources[]`, `warnings[]`

Important request/response rules

- Use stable snake_case field names in JSON. Document timestamps as ISO-8601 with timezone where possible.
- Use consistent errors: HTTP 400 invalid query, 401/403 unauthorized, 404 unknown ID, 422 invalid body, 500 unexpected server error.
- Return pagination metadata for lists. Never return tens of thousands of findings to the browser at once.
- For a record that has no ML prediction, return `prediction: null` or an explicit status, not a fabricated priority.

8. Frontend pages and expected behavior

Page / component	Display	Key interactions
Dashboard	Total findings, severity/priority distribution, top high-risk findings, cloud breakdown, last scan/data freshness	Click a KPI/chart segment to filter findings; show empty/error/loading states.
Findings table	ID, cloud, region, resource type, finding title, severity, ML priority, confidence (optional), status, detected date	Search, filter, sort, paginate; preserve filters in URL if practical.
Finding detail	Finding description, evidence, resource, recorded severity, ML prediction, contextual score if available, recommendation, sources	Copy IDs; navigate to resource or history; mark UI fields as prediction/heuristic where needed.
Resource graph	Resource nodes, relation type, direction, evidence/source, verification state	Use a distinct style for unverified/candidate links; never label a heuristic association “confirmed attack path.”
Temporal / trends	New, persisting, worsening, improved, recurring, resolved events; per-scan counts	Select time/scan; show scan completeness and warn if scans have different scopes.
Remediation tracking	Suggested steps, owner, state, applied timestamp, verification result, feedback	Update app tracking only if authorized by backend; never imply a cloud configuration was changed.
AI-SPM	AI workload assets, control statuses, evidence coverage, failure/gap summary	Show UNKNOWN distinctly; link to evidence metadata and assessment timestamp.
Fridat chat	Question input, response, citations, source files/IDs, warnings, retrieved record count	Render citation chips; show no-data warnings; never expose Gemini key in browser.

UI design rules

- Keep recorded severity, ML predicted priority, contextual risk score, and remediation state visually separate.
- Make unknown/missing data visible. “No rows returned” does not mean “no risks exist.”
- Show source identifiers on finding/graph/chatbot pages so an analyst can trace claims back to a record.
- Never let the UI calculate its own priority score or make decisions from hardcoded sample data in production mode.

9. Integrating the intelligence modules

Module	Current artifacts / data	How frontend/backend should treat it
ML priority prediction	`cspm_priority_baseline_model.pkl`, `cspm_ml_predictions.csv`	Output is a prediction learned from existing `priority` labels; show confidence with a caveat.
Contextual risk scoring	`cspm_contextual_risk_scored.csv` and `cspm_contextual_risk_summary.csv` when generated	Heuristic score: show score methodology/source and never confuse with ML confidence.
Graph / candidate correlation	`cspm_asset_graph_nodes.csv`, `cspm_asset_graph_edges.csv`, `cspm_candidate_attack_paths.csv`, `cspm_blast_radius_analysis.csv` where available	Candidate correlations should be labelled unverified. Relationship evidence is needed to confirm actual paths.
Topology evidence validation	`cspm_asset_relationships.csv`, `cspm_verified_relationship_validation.csv`, `cspm_candidate_attack_paths.csv`	Validation of metadata is not independent proof of network reachability or exploitability. Include evidence source and validation status.
Temporal monitoring	`cspm_finding_history.csv`, `cspm_temporal_events.csv`, `cspm_scan_metrics.csv` where generated	Use only real scans with stable IDs and comparable scope. Absence resolves only if the scan is complete and rules permit it.
Remediation / feedback	`cspm_remediation_tracking.csv`, `cspm_analyst_feedback.csv` where generated	Tracking status is not proof of an actual cloud fix; verified resolution needs a fresh scan/evidence.
Anomaly detection	`cspm_anomaly_detection_results.csv` where generated	Treat flags as investigation signals. A scan drop can mean scanner coverage failed.
AI-SPM	`cspm_ai_workload_inventory.csv`, `cspm_ai_spm_assessment.csv`, `cspm_ai_spm_findings.csv`, evidence-aware outputs where generated	Inventory templates are not proof of live AI assets. Keep UNKNOWN and evidence gaps explicit.
Fridat / Gemini	Gemini-backed chat notebook with CSV retrieval and citation checks	Retrieve relevant rows first; cite real file/identifier pairs; never send cloud secrets; return warnings when no tables match.

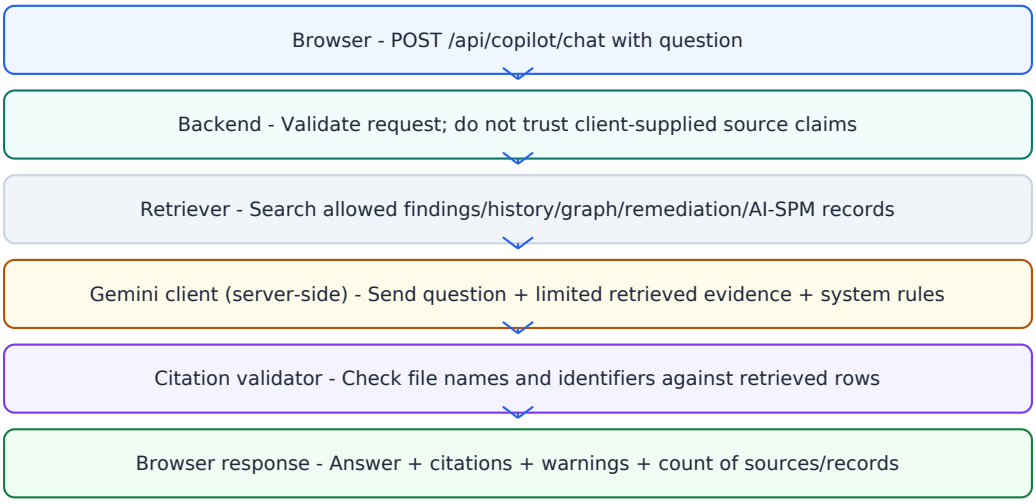
Important observed symptom: the latest screenshot showed “retrieved from 0 table(s)” plus a citation warning. Treat that as a retrieval/configuration issue to fix before presenting Fridat as connected to the project’s real data.

10. Suggested persistence model and request sequence

For a first version, use a relational database (or a clearly separated data-access layer around the current CSV outputs). Keep the canonical finding separate from derived predictions and time-dependent records.

Table / collection	Suggested key fields	Why it exists
`findings`	`finding_id` PK, `resource_id`, cloud, category, severity, evidence, status, detected_at, source_scan_id	Canonical scanner output; one stable row/record per finding identity or version, based on agreed semantics.
`ml_predictions`	prediction_id, finding_id FK, predicted_priority, confidence, prob_P1..prob_P4, model_version, created_at	Preserve prediction history and identify which model produced it.
`resources`	resource_id, cloud, region, resource_type, display_name (if safe), first_seen, last_seen	Deduplicate assets and link findings to a resource.
`relationships`	edge_id, source_resource_id, target_resource_id, type, direction, evidence_reference, source_system, verification_status, observed_at	Store explicit topology/IAM relationship evidence and its provenance.
`scan_runs`	scan_id, provider, started_at, completed_at, scope, complete_flag, status	Interpret temporal change only across comparable scans.
`temporal_events`	event_id, finding_id, prior_scan_id, current_scan_id, event_type, created_at	Audit changes across scans.
`remediation_tracking`	remediation_id, finding_id, state, owner, suggested_action, applied_at, verified_at, verification_scan_id	Track operational work; do not equate status with a cloud change.
`ai_spm_assessments`	ai_asset_id, control_id, status, evidence_reference, evidence_source, observed_at, reviewed_by, reviewed_flag	Control state plus evidence metadata and assessment coverage.
`chat_audit` (optional)	request_id, user_id, question metadata, sources used, timestamp, latency, safety outcome	Troubleshooting/audit without logging secrets or unnecessary sensitive prompt contents.

Chat request sequence



11. Recommended build order for your teammate

Phase	Frontend task	Backend task	Exit criteria
1. Agree schema	Build typed UI models from shared field contract	Normalize scanner fields; create mock response examples	One documented canonical finding JSON schema
2. Read-only findings	Overview + table + detail using mock API then real API	Implement health, overview, list, detail, filters, pagination	UI shows actual stored records; no hardcoded security claims
3. ML integration	Show predicted priority and confidence separately	Load fitted model on server; validate model-ready features; persist/version predictions	Known test row returns predictions matching the notebook pipeline
4. Analytics	Distribution charts and filters	Aggregates derived from actual records/outputs	Counts reconcile with filtered dataset
5. Graph and temporal	Graph with provenance; history timeline	Relationship/event endpoints with verification and scan metadata	Unverified edges are visually distinct; scan gaps are surfaced
6. Remediation and AI-SPM	Tracking, control status, evidence coverage screens	Read/write only to app-level tracking data as authorized; validate rescan state	UNKNOWN remains unknown; tracking does not pretend a cloud fix ran
7. Fridat	Chat panel, citation rendering, warnings	Retrieve scoped data; call Gemini server-side; validate citations; redact secrets	No-data questions return explicit limitation; citations verify
8. Hardening	Error, loading, empty, mobile/responsive states	Auth/RBAC, logs, rate limits, input checks, tests	Security tests pass; no API key in browser bundle/logs

Recommended repository layout

```
frontend/
src/pages/   (Dashboard, Findings, FindingDetail, Graph, Trends, AI-SPM, Fridat)
src/components/   (FilterBar, FindingsTable, RiskBadge, EvidencePanel, CitationList)
src/api/   (typed API client; central error handling)

backend/
app/main.py   (application entry point)
app/routes/   (overview, findings, predictions, graph, temporal, remediation, ai_spm, copilot)
app/schemas/   (request/response validation)
app/services/   (findings, risk, temporal, graph, copilot retrieval)
app/ml/   (load model, feature engineering, inference)
app/repositories/   (database/CSV access)
app/security/   (auth, redaction, audit)
```

This layout is a suggested target organization. Adapt it to the existing repository instead of moving working code blindly.

12. Acceptance checklist and testing plan

Test	Expected behavior
No data available	API returns an empty list and metadata; UI says no matching records were found, not “no issues exist.”
Unknown finding ID	API returns 404; UI provides a clear not-found state.
Filtering and pagination	Returned items/count match provider, severity, priority, status, and text filters.
Feature missing	Prediction service returns a validation error or an explicit unavailable result; it does not silently fill security flags as false.
Training/inference parity	The same known example receives the same transformed features and expected prediction from the saved pipeline.
Citation with valid row	Citation filename and identifier match a record included in retrieval.
Citation with made-up ID	Validation fails or answer is flagged; status must not claim “passed” with no sources.
Prompt injection in evidence	Text inside CSV evidence is treated as untrusted content, never as assistant instructions.
Conflicting records	Assistant surfaces source/date conflict rather than silently picking a winner.
Unverified graph edge	UI labels it candidate/unverified and shows provenance; not a confirmed attack path.
Incomplete scan	Missing findings are not marked resolved solely because they were absent.
Remediation status	“Applied” is not presented as “verified resolved” without a qualifying check/rescan.
Secret handling	Gemini key stays in Colab Secrets or server secret storage; not in JavaScript, repository, output, or logs.
Cloud write safety	Normal UI/backend flows only read cloud evidence; changes require a separate explicit, authorized workflow.

Security rules

- Never embed `GEMINI_API_KEY` in frontend code or check it into Git. In production, store it in backend secret management.
- Treat file contents and retrieved records as untrusted. Use server-side allowlisted tools; never allow the LLM to run arbitrary SQL, shell commands, or cloud APIs.
- Apply authentication/authorization, input validation, rate limiting, safe output encoding, and least-privilege access before exposing the service.
- Keep an audit trail of data sources used, model version, prediction timestamp, and status transitions where appropriate.

13. ML limitations and quick glossary

Limitations the app must communicate

• The current target is the dataset's existing `priority` value. This teaches the model to reproduce patterns in those labels; it does not automatically prove the priority is correct.

• The dataset includes synthetic data in the project workflow. Synthetic labels and generated rows are useful for prototyping but are not proof of live security exposure or real-world performance.

• The model feature-importance table is global importance, not a guaranteed explanation for a single finding.

• `ml_confidence` is the maximum predicted class probability. It should not be shown as "probability this resource is compromised."

• Candidate graph edges based on correlation groups are associations. Verified attack paths need explicit topology/IAM evidence and technical validation.

• Anomaly detection based on scan counts is not user behavior analytics or proof of an attack. Sudden drops may reflect missing scanner coverage.

• The evidence-aware AI-SPM score depends on declared control weights and reviewed evidence metadata; it is not a formal compliance certification.

Glossary

Term	Plain-English definition
CSPM	Cloud Security Posture Management: continuously inspect cloud configuration for security risks.
Finding	One detected issue with an affected resource, evidence, severity, and status.
Feature	An input field used by the ML model to make a prediction.
Target / label	The answer the model was trained to predict; here the existing `priority` column.
Inference	Using a saved trained model to predict on new data.
Preprocessing	Preparing data for the model, including missing-value handling and categorical encoding.
Random Forest	An ML algorithm that combines many decision trees to predict a class.
Confidence	A model probability-derived signal; not the same as security certainty.
Correlation group	A dataset label that associates findings; it does not prove a direct resource connection.
Provenance	Where data came from and how it was observed/verified.
AI-SPM	AI Security Posture Management: assess security controls around AI workloads and associated data/access.
RAG / retrieval	Find relevant project records first, then provide them to the LLM to ground its answer.

Definition of done for this handoff

The frontend/backend teammate should be able to build the screens and APIs without retraining or reverse-engineering the ML. That requires: an agreed finding schema, a backend inference adapter to the saved pipeline, clearly defined API responses, data provenance, and tests that separate recorded facts from predictions and heuristics.

Project reference artifacts
ML notebook outputs include `cspm_findings_feature_engineered.csv`, `cspm_priority_baseline_model.pkl`, `cspm_ml_predictions.csv`, `cspm_feature_importance.csv`, `cspm_candidate_attack_paths.csv`, and `cspm_blast_radius_analysis.csv`. Other temporal, remediation, graph, and AI-SPM outputs exist only after their corresponding modules have been run and saved.